Proof notes: what each result actually says and why it's true

Hoang Nguyen

Abstract

This is a companion to `writeup/main.tex` (the formal report, which states every definition and theorem but proves none of them in prose) and to `lean/gc/README.md` (which records *design* decisions, not proofs). This document walks the results that are actually *proved* in `lean/gc/Gc/`: what each theorem says, the shape of its Lean proof, which invariants it genuinely needs, and — since these are finished mechanized proofs rather than paper sketches — the lemmas discovered along the way that turned out to be reusable, and the places where the argument has a real wrinkle a paper proof would gloss over. Where `main.tex` already states a definition or theorem precisely, we cite it by name instead of restating it, and spend the space on what `main.tex` does not have: the actual proof.

Scope: `Gc/Model/` and `Gc/Reachability/Reachable/`. `Gc/Reachability/Referencable/` is deliberately excluded — it is deprecated and superseded by `Reachable/` (see `CLAUDE.md`).

Contents

1 `Gc/Model`: the operational semantics and its invariants

1.1 The model and its rules

The configuration $\langle S, H\rangle$ (stack of frames, heap of regions) and the nine mutation operations' operational-semantics rules are given in full in `writeup/main.tex`, in its "Model" and "Operational Semantics" sections.[1] We reuse that notation throughout: $\mathsf{Rld}(rid)/\mathsf{Old}(oid)$ for references, $a.loc(\langle S, H\rangle)$ for where a reference resolves ($\mathsf{Stk}(fid)$ or $\mathsf{Rgn}(rid)$ or undefined), $a.objAt(\langle S, H\rangle)$ for the object an Old reference currently names.

1.2 The ten invariants of *ValidConfig*

`main.tex`'s "Invariants" section defines all ten (*L1*, *L2*, *H1*, *H2*, *H3*, *S1*, *S2*, *S3*, *HS1*, *HS2*) and bundles them into *ValidConfig*. We do not restate the formulas; the table below is only a one-line reminder of what each one *forbids*, since that is the fact every proof below actually reaches for.

Invariant	What it rules out
L1	the same object id allocated twice (stack or heap, anywhere)
L2	a frame executing inside a region that is actually *Closed*
H1	a region whose bridge object isn't actually a member of its own *objMap*
H2	two live $\mathsf{Rld}(rid)$ references to the same region
H3	a region's contents holding a reference that resolves *outside* that region
S1	two stack frames both claiming the same region
S2	a frame referencing a stack object owned by a *later* frame
S3	a frame referencing a region owned only by a *later* frame
HS1	a dangling Old (points at an id never/no-longer allocated)
HS2	a dangling Rld (points at a region no longer in the heap)

Remark 1.1. *HS1*/*HS2* are not derivable from the rest: *H2*, *H3*, *S2*, *S3* all presuppose a reference that *already* resolves somewhere; none of them constrains a reference that resolves to nothing. Without *HS1*/*HS2*, a stale reference could sit inert and then coincidentally collide with a freshly allocated id later, and nothing above would have caught the staleness in between. (`main.tex` makes the same point; it surfaced while proving *S2*/*S3* preservation for the three `make*` operations, whose `freshObjectId`/`freshRegionId` only look at ids currently used as *keys*.)

Four corollaries proved directly from the invariants (`Validity.lean`) are cited constantly by everything downstream, in both layers:

- `oid_in_stack_implies_not_in_heap` / `oid_in_heap_implies_not_in_stack` — from *L1*: an id found by one search can't also be found by the other.
- `oid_loc_rgn_iff_in_heap`, `oid_loc_stk_iff_in_stack`, `rid_loc_rgn_iff_in_heap` — rewrite the opaque computation $\mathsf{Old}(oid).loc(\langle S, H\rangle) = \mathsf{Rgn}(rid)$ into the usable $\exists\, region.\ H(rid) = region \wedge oid \in region.objMap$. Every later file that needs to reason about *where* an id lives goes through one of these three, never through the raw definition of $.loc(\cdot)$.

[1] This document compiles standalone, so cross-file `\ref`s to `main.tex`'s labels are not used; section names are given in prose instead.

1.3 *RuntimeConfig.start* satisfies ***ValidConfig***

One root region (Open, bridge object 0) and one root frame executing in it. Each of the ten invariants reduces, by unfolding, to a fact about a one- or two-element list, closed by `decide/simp/List.nodup_singleton`. No interesting content — this file exists purely to anchor the induction below.

1.4 The preservation proof pattern

Every `Gc/Model/Mutation/<Op>.lean` defines the operation as an *Option RuntimeConfig*-valued `do`-block (each bind is a precondition that can fail — exactly the premises of the corresponding inference rule in `main.tex`), paired with an `<op>_cases` lemma that unpacks a successful run into an explicit disjunction of fully-instantiated hypotheses, one disjunct per inference-rule variant. This is mechanical but load-bearing: it is, in effect, the operation's real specification — a mistake here (a missing disjunct, a wrong side condition) would not surface as a Lean error in its own short proof (which just replays the `do`-block's control flow); it would surface only as an unprovable, or vacuously provable, downstream theorem. Every proof from here on, in both layers, destructs a hypothesis $op\ cfg = \mathsf{some}\ cfg'$ via `<op>_cases` exactly once, then argues with concrete named values for the rest.

`Preservation/<Op>.lean` then proves ten lemmas `<op>_L1`, …, `<op>_HS2` and combines them:

$$\texttt{<op>\_valid} : \mathit{ValidConfig}(cfg) \to op\ cfg = \mathsf{some}\ cfg' \to \mathit{ValidConfig}(cfg')$$

`Preservation.lean` aggregates all nine into *AllPreserve*(*ValidConfig*) by case-splitting on the reified *Stmt* type (`Mutation/Stmt.lean`) — the statement "every one of the nine operations preserves *ValidConfig*" proved once, generically, rather than left as nine theorems a caller has to know to look up individually.

1.5 Per-operation notes

For each operation, the genuinely interesting invariant(s); the rest are short "nothing relevant changed" arguments.

`varAsgn x yf` Two branches on whether `x` is the frame's own bridge variable. If so, this is really a *bridge-object reassignment* — subtle because it changes *H1*'s witness for that region without touching $objMap$ at all; the proof shows the *new* bridge id is still a key of the (unchanged) $objMap$, which falls straight out of the rule's own side condition ($yfRef.loc(\cdot) = \mathsf{Rgn}(rid)$ with $rid = frame.regionId$ — you can only rebind the bridge to something already inside the same region). If not the bridge var, it's a plain local write gated by $resolveV(\mathtt{x}, \cdot) = \epsilon$ (no existing binding), which exists purely to keep *resolveV* (outward frame search) unambiguous.

`fieldAsgn xf y` The `field-asgn-stack/field-asgn-region` split is the clearest illustration of why *H3* needs an explicit *status* = *Open* side condition: a region write is permitted only while the writer's frame executes *in* that exact region *and* the new value resolves into the *same* region. Drop either equality and *H3*'s preservation becomes false — nothing else in the rule would stop the region from acquiring an outward-pointing reference.

`swap x yf` Syntactically the busiest rule (two locations get new values simultaneously) but its invariant proofs are `fieldAsgn`'s field-write argument and `varAsgn`'s var-write argument run side by side — no new invariant-level content.

`makeObjStack x` / `makeObjRegion x` Allocate a fresh, empty object at $\langle S, H\rangle.freshObjectId$. *L1* preservation is exactly `RuntimeConfig.freshObjectId_not_mem` (`Helpers.lean`): the fresh id is, by construction, not already in $cfg.objectIds$. *H2*/*HS1* for the new id are vacuous (nothing points at it yet). `makeObjRegion` additionally needs $status = Open$ — an object can only be born directly inside a region while someone is executing there.

`makeRegion x` Allocates a brand-new *Closed* region with its own fresh bridge object, and stores an RId to it in the current frame. *H2* for the new region id is again vacuous by freshness (`freshRegionId_not_mem`); starting *Closed* is what keeps *L2* trivially true — nobody is executing inside the new region yet.

`merge x` By far the busiest file (∼1200 lines, `Preservation/Merge.lean`), being the only operation that deletes a heap key and combines two $objMap$s into one. The load-bearing fact, proved first and reused by all ten invariant proofs, is that the two regions being merged have *disjoint* object-id sets:

Lemma 1.2 (disjointness of the merged regions)**.** If $H(rid) = region$, $H(rid') = region'$, $region.status = Open$, $region'.status = Closed$, and $\langle S, H\rangle \vdash L1$, then $rid \neq rid'$ (Open/Closed are mutually exclusive) and $dom(region.objMap) \cap dom(region'.objMap) = \emptyset$ (else *L1* would be violated).

Disjointness turns the potentially lossy $AList.union$ (which can silently drop/reorder colliding keys in general) into a plain, permutation-preserving append, and every subsequent *L1*/*H2*-style counting argument goes through by permutation-invariance of `List.count`/`Nodup`. A second recurring fact re-derives "an id lives in at most one heap entry" for the *post*-merge heap, by case-splitting on whether each of two witnessing entries is the merged head or came from the untouched remainder (which embeds back into the original heap via a `List.Perm` argument built from two nested `List.exists_of_kerase` extractions). `merge` is also the one operation whose *reachability*-layer proof (§2.4) needs a genuinely different idea from the other eight, because it is the only operation that deletes a region and a live RId reference along with it.

`enter xf bridgeVar` / `exit` Push/pop a frame, flipping the entered/exited region's status. *L2*/*S1* are index bookkeeping; the freshly pushed frame starts with empty $objMap/varMap$, so *S2*/*S3*/*HS1* need say nothing new about it — its only content is the just-flipped bridge object, already covered by the region-side invariants.

1.6 Reusable toolkit: `Preservation/Common.lean`

Five generic lemmas, factored out once the same fact was being reconstructed byte-for-byte in several files:

- `stackWithIndex_frame_transport_down_of_shape_eq` / `_up_of_shape_eq` — if cfg, cfg' agree up to some projection $\pi : Frame \to \alpha$ at every stack *position*, $stackWithIndex$ membership transports between them, preserving $.index$ and the projected value. Generic in π: a proof that only cares about $(regionId, bridgeVar)$ can ignore $objMap/varMap$ changes entirely.
- `stackWithIndex_objMap_get_eq_of_last_varMap_update` — the common special case "only the last frame's $varMap$ changed" $\Rightarrow$ $objMap$ is unaffected at every position, indexed or not. Used by every pure var-write operation.

- `merge_corollary_regionId_unique_index` — *S1* restated at the $FrameWithIndex$ level (two frames sharing a *regionId* are the same frame). Despite the name (kept from wherever it was first needed) this is used throughout §2.2 to convert "same region" facts into "same index" facts, which is what most confinement arguments actually need.
- `swap_corollary_stackWithIndex_index_inj` / `_find_eq` — two frames with the same *.index* are the *same* element (since *.index* is assigned by position, not stored data). Arguably the single most-cited lemma in `Gc/Reachability/Reachable/` — see §3.

1.7 Pitfalls specific to this layer

Pitfall 1.3 (`AList.insert` reorders on an existing key)**.** Inserting at an already-present key goes through *kinsert*/*kerase* and can reorder *.entries*, unlike inserting at a genuinely fresh key. Any proof needing *.entries*-level equality after an *insert* (not just *.lookup*-level) — `merge`'s region-combining argument above — has to go through an explicit `List.Perm` argument (`exists_of_kerase` + `NodupKeys.eq_of_mk_mem`), never assume "insert = append."

Pitfall 1.4 (*Status* has no `LawfulBEq`)**.** *Status* derives BEq/$DecidableEq$ but not $LawfulBEq$, so `beq_iff_eq` doesn't fire on it. Every "Open xor Closed" contradiction in this codebase is closed by case-splitting *region.status* directly and `decide`, never by rewriting a `==` hypothesis.

Pitfall 1.5 (the `<op>_cases` lemma *is* the semantics)**.** Because the operation is opaque `do`-notation, when in doubt about an operation's actual precondition, read its `<op>_cases` lemma's statement, not the `do`-block — a mistake in the former is invisible until a downstream theorem quietly stops being provable.

2 `Gc/Reachability/Reachable`: the reachability layer

2.1 What's already in `main.tex`

`main.tex`'s "Reachability" section already gives, in full, the definitions of *ReachableStep* ($a \to b$), *RegionReachable*, *FrameReachable*, *StackReachable*, *FrameRoot*; the two "is exactly the reflexive-transitive closure of $\to$" theorems; and (its "Suspended regions" subsection) *FRALF* together with its single-step preservation theorem and the headline suspended-region stack-reachability invariant. We do not restate any of these; we only fix the two facts about *ReachableStep* that everything below leans on, then move straight to how the theorems are actually proved.

Remark 2.1 (the one thing RId can do)**.** $\mathsf{RId}(rid) \to b$ only when $H(rid).status = Closed$, in which case b is forced to be $\mathsf{OId}(H(rid).bridgeObjectId)$ — a Closed region's RId has exactly one outgoing step, into its own bridge object; an *Open* region's RId has none (`open_rid_no_step`, from *L2*: if a region's RId could step it would have to be Closed, contradicting an on-stack frame owning it). This one asymmetry — the "portal" into a suspended region — is the entire reason this layer exists as a sibling to the deprecated $RefStep$ rather than a refinement of it, and it is what makes §2.3 non-trivial.

Remark 2.2 (why the $\xrightarrow{*}$-reformulation matters)**.** *RegionReachable*/*FrameReachable* are small inductive relations, but almost every nontrivial proof below inducts on their `Relation.ReflTransGen`-reformulation instead (the "is exactly the transitive closure" theorems from `main.tex`), never on the raw inductive constructors. `ReflTransGen` comes with two mathlib induction principles: ordinary constructor induction (fixes the chain's *first* element, grows the *last* by one step at a time — the right shape for "trace forward from a known root") and `head_induction_on` (fixes the *last*

element, peels off the *first* step — the right shape for "chase an escaping chain hop by hop from a known start"). Both get used below, for different lemmas; picking the wrong one does not produce a type error, it produces a stuck goal whose motive silently doesn't generalize the variable the argument actually needs to induct on.

2.2 The confinement toolkit (`Lemmas/Common.lean`)

This file is the layer's real intellectual core. Every per-operation proof in `FrameReachableAtLaterFrame/` and `StackReachableInvariant/` cites several of these directly.

Lemma 2.3 (backward confinement, Open region)**.** (`predecessor_of_region_object`) If $\mathsf{Old}(oid).loc(\cdot) = \mathsf{Rgn}(rid)$ and $H(rid).status = Open$, then any a with $a \to \mathsf{Old}(oid)$ is itself $\mathsf{Old}(oid')$ for some oid' with $oid'.loc(\cdot) \in \{\mathsf{Rgn}(rid)\} \cup \{\mathsf{Stk}(fid) \mid fid\}$ — never a bare Rld, never an object of a *different* region.

Proof idea. Case on a. If $a = \mathsf{Rld}(rid')$: either $rid' = rid$, ruled out directly by `open_rid_no_step` above, or $rid' \neq rid$, in which case the only possible step target is rid''s own bridge object, whose location is $\mathsf{Rgn}(rid')$ by *H3*, contradicting $\mathsf{Rgn}(rid)$. If $a = \mathsf{Old}(oid')$: *H3* applied to whatever region oid' turns out to live in forces its location to be $\mathsf{Rgn}(rid)$ too (region case), or it's already on the stack (stack case) — never a third region. □

Lemma 2.4 (backward confinement, Closed region)**.** (`predecessor_of_closed_region_object`) Same statement with *Closed* in place of *Open*, except the "on stack" branch becomes *impossible* instead (*L2*'s contrapositive: an on-stack frame can never own a Closed region — `closed_region_not_owned`), and the Rld branch becomes the desired *portal hop* instead of a contradiction: $a = \mathsf{Rld}(rid)$ with $oid = H(rid).bridgeObjectId$ is now an admissible predecessor.

Lemma 2.5 (safety and its transport)**.** Define $SafeRef_{rid}(ref)$ for $ref = \mathsf{Old}(oid)$ as $oid.loc(\cdot) = \mathsf{Rgn}(rid)$ or $oid.loc(\cdot) = \mathsf{Stk}(fid)$ for some fid (never true for a bare Rld). Then:

1. (`predecessor_safe`) safety propagates *backward* across one step, by Lemma 2.3;
2. (`safe_reflTransGen_transport`) if $\to_{cfg}$ and $\to_{cfg'}$ agree on every Old-sourced step, any cfg-chain ending at a *SafeRef* target transports wholesale to a cfg'-chain.

This pair is exactly the tool used for every operation whose mutation changes no existing object's *content* — see the "additive" case in §2.4.

Lemma 2.6 (index confinement)**.** (`stack_container_confined`, `region_container_confined`) If $\mathsf{Old}(oid_C)$ is stack-resident at index fid_C (resp. resides in an *Open* region owned by frame index fid_C), then anything $FrameReachable(fid, \mathsf{Old}(oid_C))$ has $fid \geq fid_C$.

Proof idea. Structural induction on the *FrameReachable*-witness's `ReflTransGen` chain, generalized over the chain's *end* (ordinary induction, not `head_induction_on` — the claim is about the chain's *root*, which the inductive step naturally exposes). Each step either stays inside the confined container (index unchanged, recurse) or escapes onto the stack, at which point the one-hop bound $S2/S3$-at-the-`ReachableStep`-level lemmas (`stack_step_index_le`, `stack_step_region_index_le`) fire directly. □

This is the mechanized form of "you can't reach backward in time", and it is what every per-operation proof in both §2.4 and §2.5 calls to turn "was the mutated container already visible from here" into a plain numeric comparison.

Lemma 2.7 (Closed-region confinement — the engine behind §2.3)**.** (`RegionReachable_of_-FrameReachable_closed`, `FrameReachable_rid_of_closed_container`) If $H(rid).status = Closed$ and $oid.loc(\cdot) = \mathsf{Rgn}(rid)$, then *any* $FrameReachable(fid, \mathsf{OId}(oid))$ witness is, in fact, $RegionReachable(rid, \mathsf{OId}(oid$ *and* it forces $FrameReachable(fid, \mathsf{RId}(rid))$ too — reaching anything inside a Closed region is never possible except by first reaching the region's own portal.

Proof idea. Backward induction over the chain, dispatching at each step on Lemma 2.4's two cases: either continue tracing inside the region, or hit the portal hop and stop. □

2.3 The *CR6* theorems

Not stated in `main.tex`; proved directly from Lemma 2.7, no new technique.

Theorem 2.8 (*CR6*)**.** Let $H(rid).status = Closed$ and $oid.loc(\cdot) = \mathsf{Rgn}(rid)$.

1. If $\langle S, H\rangle \vdash SReachable(\mathsf{RId}(rid))$, then $SReachable(\mathsf{OId}(oid)) \iff RReachable(rid, \mathsf{OId}(oid))$.
2. If $\langle S, H\rangle \nvdash SReachable(\mathsf{RId}(rid))$, then $\langle S, H\rangle \nvdash SReachable(\mathsf{OId}(oid))$ for *every* such *oid*.

Part 1's forward direction doesn't even need the RId-reachability hypothesis (closedness alone forces the single-portal argument); the hypothesis is needed only to glue the portal hop onto an already-stack-rooted chain in the reverse direction. Part 2 is (1)'s contrapositive, given its own name because it is the practically useful reading: an unreachable Closed region's contents are *all* unreachable, full stop.

2.4 Proving *FRALF* preservation: three proof shapes

FRALF (defined in `main.tex`, see above) is proved to hold vacuously at $RuntimeConfig.start$ (a single frame makes $F.fid < F'.fid$ unsatisfiable), then single-step-preserved by all nine operations. The nine proofs split into three genuinely different argument shapes.

1. Additive / status-only operations — `Enter`, `Exit`, `makeObjStack`, `makeObjRegion`, `makeRegion`, `varAsgn`. No existing object's *content* changes. The workhorse is Lemma 2.5(2): show the tracked object is *SafeRef*, show the step relation agrees on every OId-sourced step (trivial when nothing's content changed), and the whole chain transports for free — no per-hop case analysis. `Enter`'s proof is the most involved member of this group only because pushing a frame means the "later frame" in the *FRALF* statement might now instead *be* the freshly pushed one, whose only possible root is the entered region's own bridge object; a short safety argument shows that can't equal the tracked object in the (different) suspended region.

2. Content-mutating operations — `fieldAsgn`, `swap`. These genuinely rewrite one existing reference slot (call the mutated object's id oid_C), so the step relation changes at exactly that one point and Lemma 2.5's uniform transport no longer applies. The proof instead:

1. shows oid_C's *owner* index is a hard lower bound on anything that can reach it (Lemma 2.6, now applied to the mutated object rather than the tracked one) — call this bound fid_{active};
2. for any chain confined strictly below fid_{active}, the step relation is untouched, so it transports for free;
3. for the one remaining case — a chain rooted exactly at the mutating frame, which *might* use the new edge — walks the chain forward via `head_induction_on`, splitting at each hop on whether it is the newly-written one. If it is, the chain escapes onto the field's *new* value, which has its own independent root (`resolveV_frameRoot`/`resolveFA_frameReach`), and the argument reduces back to case 2.

This is exactly `CLAUDE.md`'s "owner-index bound turns 'could this new edge matter' into a decidable dichotomy, resolved by chasing the new value's own independent root", traced through concretely.

3. `merge` — its own shape. `merge` is the only operation that *deletes* a live $\mathsf{RId}(rid')$ reference along with the region it names. The key trick:

Lemma 2.9 (an about-to-be-deleted region reference is never a step *target*)**.** (`merge_ridPrime_not_step_target`) If $\mathsf{RId}(rid')$ occurs live in $\langle S, H\rangle$ (as it must, to be merged), no reference steps *into* it: any putative predecessor would push *H2*'s count for rid' to 2, contradiction.

Consequently $\mathsf{RId}(rid')$ can only ever be a chain's *root*, never mid-chain, so "does this chain use the deleted reference" is answered by inspecting the root alone, never by a hop-by-hop induction. This gives a chain-avoidance transport (`merge_chain_transport_down`: any $\mathsf{RId}(rid')$-avoiding chain transports unconditionally $cfg' \to cfg$) plus one genuine escape case, handled by

Lemma 2.10. (`merge_ridPrime_reflTransGen_iff`) A $\mathsf{RId}(rid')$-rooted chain to any target $\neq \mathsf{RId}(rid')$ is the same chain, one hop shorter, rooted at $H(rid').bridgeObjectId$ instead — since $\mathsf{RId}(rid')$'s only possible first hop is that fixed bridge object.

2.5 The suspended-region stack-reachability invariant

The headline theorem (stated in `main.tex`, see above) is stated as an $\iff$, not a one-way implication, and takes *FRALF* as an *explicit hypothesis* rather than folding it into a *ValidConfig*-style bundled invariant — this keeps its own proof independent of whether *FRALF* has separately been shown inductive, at the cost of every caller needing to supply it.

Each per-operation proof in `StackReachableInvariant/` uses *exactly* the same case analysis as its §2.4 sibling — compare, e.g., the two `fieldAsgn` files: same lemmas, same split on whether the frame in question is the active one, same escape argument — because "was *oid* reachable before" and "is it reachable after" are genuinely symmetric questions once confined transport exists in both directions. The forward direction of the $\iff$ is close to a direct instance of *FRALF*-style transport; the *backward* direction is where the escape-chain argument (§2.4, cases 2/3) actually earns its keep, since going from cfg' back to cfg is where a chain might have used the newly-written edge.

Pitfall 2.11 (an $\iff$ needs both halves proved independently)**.** It is tempting to think the $cfg' \to cfg$ direction of the escape argument is just the $cfg \to cfg'$ direction "with the arguments flipped". It is not: the newly-written edge exists in cfg' but not cfg, so the escape case genuinely only arises in one of the two directions and has to be proved on its own terms there.

2.6 What's deliberately not done here

No analogue of a `Guarantees.lean`-style top-level GC-safety theorem sits on top of *StackReachableInvariant* yet. Everything above is a *single-step* fact; composing it over an arbitrary-length trace — the literal `main.tex`/report.pdf claim, which talks about invariance for as long as a region stays suspended, not just across one operation — is not currently being pursued.

3 Lemmas load-bearing across both layers

- `swap_corollary_stackWithIndex_index_inj` — "same index ⇒ same element" for $stackWithIndex$. Trivial to state, indispensable once a proof juggles three or four differently-named frame variables that are provably-but-not-syntactically the same frame (a recurring situation throughout §2.4's per-operation files).

- Lemma 2.6 — turns *every* "can this later/mutated thing be reached from there" question into a numeric comparison, which is what makes the case splits in §2.4 decidable rather than requiring genuine reachability search.
- The "*H2* forbids a reference from being a step target" trick (§2.4) generalizes beyond `merge`: any future operation that *deletes* a uniquely-referenced region reference can reuse the same shape (count-based non-membership ⇒ chain-position restriction ⇒ transport reduces to a root-only case split).
- The $\xrightarrow{*}$-reformulation theorems (`main.tex` §5) exist purely so nothing downstream ever inducts on the custom inductive relations directly.

4 Pitfalls specific to reachability-confinement arguments

(Complements, does not repeat, `CLAUDE.md`'s general "Proof-engineering gotchas" list — these are specific to the confinement style of Part 2.)

Pitfall 4.1 (Open-only tools misapplied to a Closed region)**.** Lemma 2.6 (region variant) takes $status = Open$ as a hypothesis, because its proof leans on Lemma 2.3, which is stated only for the Open case (an object inside a *Closed* region has no stack-escape branch to bound in the first place). For a Closed region the right tool is Lemma 2.7/§2.3 instead. Reaching for the Open-only lemma on a Closed region fails to apply outright rather than silently giving a wrong bound — but it is an easy mismatch when adapting an existing per-operation proof to a new operation.

Pitfall 4.2 ($loc(\cdot)$ is treated as opaque)**.** The raw definition of $a.loc(\langle S, H\rangle)$ (nested reverse-stack-search / heap-search with a match forcing exactly one to succeed) is unfolded exactly once, in `Validity.lean`, to prove the three `_iff_in_*` restatement lemmas of §1.2. Every later file treats $.loc(\cdot)$ as an opaque function characterized by those iffs, never by unfolding its definition again.